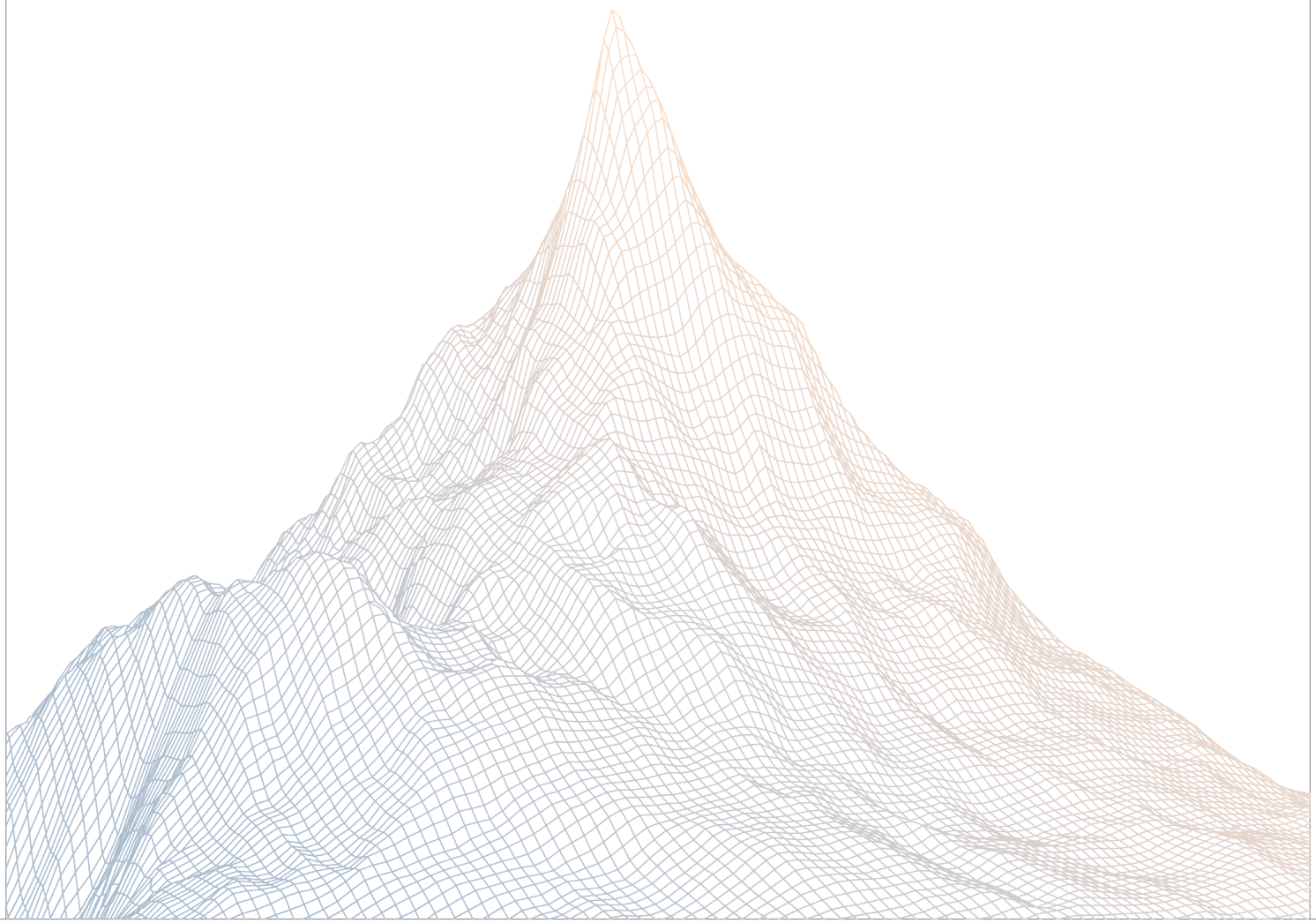
Zenith

Flex Labs

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 16th to March 17th, 2026

AUDITED BY: IIIIIII
ether_sky

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Flex Labs 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 6

4.1 Medium Risk 7

4.2 Low Risk 9

4.3 Informational 11

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Flex Labs

On-chain trading automation protocol. Modular bots for grid, DCA, vaults, and arbitrage. Programmable, composable, powered by $LBOT.

2.2 Scope

The engagement involved a review of the following targets:

Target	lq-core
Repository	https://github.com/Flex-Community/lq-core.git
Commit Hash	`0d6e454d94777bd767d14e4b43bdc4056193fd0f`
Files	`src/**/*.sol`

2.3 Audit Timeline

March 16, 2026	Audit start
March 17, 2026	Audit end
March 19, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	2
Informational	3
Total Issues	**6**

3

Findings Summary

ID	Description	Status
M-1	Type mismatch when passing cloid to cancelOrderByCloid	Resolved
L-1	Ownable2StepUpgradeable is never initialized	Resolved
L-2	The BACKEND_ROLE role can extract value via trades	Acknowledged
I-1	Incorrect function comment in transferPerpToSpot	Resolved
I-2	Type inconsistency between function parameter and event field in TakeFeeSpotToSpot	Resolved
I-3	The caller's role is incompletely verified during initialization	Acknowledged

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Type mismatch when passing cloid to cancelOrderByCloid

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- LqTrader.sol

Description:

The function `closeOrdersByCloid` accepts `cloids` as `uint64`, but passes each value directly to `cancelOrderByCloid`.

```
function closeOrdersByCloid(uint32[] calldata assets, uint64[]
    calldata cloids) external override onlyCloser {
    require(assets.length == cloids.length, "Length Mismatch!");
    for (uint256 i = 0; i < assets.length; i++) {
▷       CoreWriterLib.cancelOrderByCloid(assets[i], cloids[i]);
    }
}
```

However, the target function requires the `uint128`:

```
▷   function cancelOrderByCloid(uint32 asset, uint128 cloid) internal {
        coreWriter.sendRawAction(
            abi.encodePacked(uint8(1),
    HLConstants.CANCEL_ORDER_BY_CLOID_ACTION, abi.encode(asset, cloid))
        );
    }
```

In HyperCore, the `cloid` is uint128. Therefore, using `uint64[]` in `LqTrader` silently truncates the domain of valid `cloid` from 128 bits → 64 bits. As a result, any `cloid` values

above `2^64-1` could never be passed through this function.

Recommendations:

Update `closeOrdersByCloid()` function to accept `uint128`.

```
function closeOrdersByCloid(uint32[] calldata assets, uint64[] calldata
    cloids) external override onlyCloser {
function closeOrdersByCloid(uint32[] calldata assets, uint128[] calldata
    cloids) external override onlyCloser {
    require(assets.length == cloids.length, "Length Mismatch!");
    for (uint256 i = 0; i < assets.length; i++) {
        CoreWriterLib.cancelOrderByCloid(assets[i], cloids[i]);
    }
}
```

Flex Labs: Resolved with `@18adac15c2 ...`

Zenith: Verified.

4.2 Low Risk

A total of 2 low risk findings were identified.

[L-1] `Ownable2StepUpgradeable` is never initialized

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/LqAccessManager.sol
- src/LqConfig.sol
- src/LqFactory.sol
- src/LqFeeTaker.sol

Description:

The `Ownable` portions of the contracts are properly initialized via calls to `__Ownable_init()`, but the `2Step` portions never have their `__Ownable2Step_init()` functions called. This function is currently is a no-op so there are no negative effects, but if this changes in the future, there may be problems during an upgrade.

Recommendations:

Call `__Ownable2Step_init()` in each listed contract's initializer function

Flex Labs: Resolved with `@cce93c1d24 ...`

Zenith: Verified.

[L-2] The `BACKEND_ROLE` role can extract value via trades

`SEVERITY`: Low	`IMPACT`: High
`STATUS`: Acknowledged	`LIKELIHOOD`: Low

Target

- src/LqTrader.sol

Description:

One of the explicit goals is to prevent the backend wallet from obtaining funds. While the code prevents direct transfers of funds out of the system by that wallet, the backend wallet can still obtain funds by creating and using API/agent wallets. A backend wallet could use an API wallet to make the user enter into disadvantageous trades, with heavy slippage or at extremely poor prices, while taking the other side of the trade. Alternatively, if the user has a known strategy, the backend could maliciously over-write the existing wallet, such that the strategy fails at the most inopportune time (e.g. just prior to a large liquidation that the backend is on the other side of).

Recommendations:

Only allow the user to create API wallets

Flex: Acknowledged. The backend already has the same level of access via existing API keys, including the ability to construct and submit transactions, which is a fundamental requirement for the system to operate. Restricting the creation of new API wallets does not change this, as both backend role keys and API wallet keys exist within the same "space" and provide equivalent capabilities.Additionally, restricting API wallet creation only to users would create operational risks, since API Wallets may expire and users may be offline, while the backend still needs to manage positions in a timely manner.

For these reasons, we view this as a known design trade-off rather than a vulnerability.

4.3 Informational

A total of 3 informational findings were identified.

[I-1] Incorrect function comment in transferPerpToSpot

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- LqTrader.sol

Description:

The documentation comment for `transferPerpToSpot` incorrectly describes the transfer direction.

```
/**
 * @notice Transfer from Spot to Perp
 * @param coreAmount Amount to transfer
 */
function transferPerpToSpot(uint64 coreAmount) external override onlyUser {
    uint64 usdcPerpAmount = HLConversions.weiToPerp(coreAmount);
    CoreWriterLib.transferUsdClass(usdcPerpAmount, false);
    emit TransferPerpToSpot(coreAmount, usdcPerpAmount);
}
```

The comment states:

> "Transfer from Spot to Perp"

However, the transfer direction is Perp → Spot.

Recommendations:

Update comment properly.

Flex Labs: Resolved with `@0595920a16 ...`

Zenith: Verified.

[I-2] Type inconsistency between function parameter and event field in TakeFeeSpotToSpot

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- ILqTrader.sol

Description:

The function `takeFeeSpotToSpot` emits the `TakeFeeSpotToSpot` event using `coreAmount`, which is defined as `uint64`, while the event expects the `amount` parameter to be of type `uint256`.

```
function takeFeeSpotToSpot(uint64 tokenId, uint64 coreAmount)
    external override onlyFeeTaker {
    address feeWallet = ILqConfig(config).feeWallet();
    require(feeWallet ≠ address(0), "LqTrader: feeWallet zero");
    CoreWriterLib.spotSend(feeWallet, tokenId, coreAmount);
▷   emit TakeFeeSpotToSpot(feeWallet, tokenId, coreAmount);
}
```

Event definition:

```
event TakeFeeSpotToSpot(address indexed feeWallet, uint64 token,
    uint256 amount);
```

Solidity automatically widens `uint64` to `uint256` when emitting the event, so this does not cause a runtime issue. However, the mismatch introduces inconsistency between the function parameter type and the event field type.

Recommendations:

Change the event type to `uint64`.

```
event TakeFeeSpotToSpot(address indexed feeWallet, uint64 token, uint256
```

```
    amount);
event TakeFeeSpotToSpot(address indexed feeWallet, uint64 token, uint64
    amount);
```

Flex Labs: Resolved with `@ce68f5dc2a ...`

Zenith: Verified.

[I-3] The caller's role is incompletely verified during initialization

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- src/LqFactory.sol
- src/LqTrader.sol

Description:

Both the `LqFactory` and the `LqTrader` contracts call the `hasRole()` function on their respective `accessManagers` during initialization, but they never verify that the return value is true.

Recommendations:

Convert the raw statement to a `require()`

Flex: Acknowledged. The call to `hasRole()` (the same as the calls to `feeWallet()`) during initialization is not intended to validate the returned value, but to ensure that the provided address is a valid contract implementing the expected interface.A successful call guarantees that the address is not an EOA and that it exposes the required hasRole() function with the correct signature. This serves as a lightweight validation of the accessManager dependency rather than a role check.

Therefore, the return value is intentionally not enforced, as the purpose of this call is interface validation, not access control verification.